# Development Guidelines

Version: 1.0
---
# Purpose

This document defines the mandatory development workflow for Trading Platform Pro and its primary application, the Trading Cockpit.
The objective is to ensure:
• correct implementation
• architecture consistency
• deterministic behaviour
• trading workflow safety
• high software quality
• synchronized documentation
• long-term maintainability
Development shall follow the documented architecture and explicit capability ownership.
---
# Development Philosophy

Every change should improve the product without introducing unnecessary complexity.
Prefer:
• simplicity
• readability
• maintainability
• consistency
• explicit behaviour
• deterministic state
• small focused changes
Avoid:
• unnecessary complexity
• speculative implementations
• duplicated code
• hidden side effects
• undocumented behaviour
• generic infrastructure for hypothetical future requirements
The smallest correct change is preferred over the largest reusable abstraction.
---
# Standard Development Workflow

Every implementation follows this sequence:
1. Understand the requirement.
2. Inspect the current implementation.
3. Review affected documentation.
4. Identify architecture boundaries and dependencies.
5. Evaluate business and technical side effects.
6. Evaluate PAPER and LIVE impact where relevant.
7. Design the smallest meaningful change.
8. Implement.
9. Add or update automated tests.
10. Update documentation where affected.
11. Run focused tests.
12. Run required regression tests.
13. Run source quality checks.
14. Validate documentation generation where documentation changed.

15. Review the complete change.
16. Commit.
17. Push.
No required step shall be skipped.
Documentation updates belong to the implementation and occur before final verification, commit and push.
---
# Before Writing Code

Before implementation:
• understand the requirement
• inspect the affected source files
• inspect direct callers
• inspect direct dependencies
• review related tests
• review related documentation
• identify architecture ownership
• identify external side effects
• identify persistence impact
• identify runtime impact
• identify PAPER and LIVE impact where relevant
Do not implement from filenames, assumptions or outdated architecture knowledge.
The current repository implementation is authoritative for code-level analysis.
---
# Definition of Analysis

Implementation shall not start before the affected workflow is understood.
Analysis shall identify:
• business requirement
• owning capability
• affected modules
• direct callers
• direct dependencies
• state transitions
• external side effects
• persistence changes
• asynchronous behaviour
• failure behaviour
• test impact
• documentation impact
For trading-critical changes additionally evaluate:
• order submission impact
• duplicate submission risk
• broker acknowledgement semantics
• retry behaviour
• reconciliation impact
• PAPER and LIVE impact
Analysis is mandatory.
---
# Current Implementation First

Always inspect the current implementation before proposing code changes.
Do not assume:
• file paths
• class names
• method names

• signatures
• configuration fields
• database models
• runtime services
from documentation alone.
Documentation defines intended architecture and behaviour.
The current source tree defines the implementation that must be changed.
When documentation and implementation differ, identify the discrepancy explicitly before changing code.
---
# Smallest Meaningful Change

Every implementation should solve one clearly defined problem.
Prefer:
• one capability
• one workflow correction
• one explicit refactoring objective
Avoid combining unrelated changes.
A change should minimize:
• affected modules
• side effects
• migration effort
• regression surface
Do not redesign adjacent capabilities without a concrete requirement.
---
# Architecture Ownership

Before changing code identify the owning architecture layer.
```text
Presentation
↓
Application
↓
Domain
Infrastructure
↓
Application / Domain Ports
```
Typical ownership:
```text
Domain
→ business rules and invariants
Application
→ use cases and workflow coordination
Infrastructure
→ technical integrations and persistence
Presentation
→ UI and presentation state
```
Do not place behaviour in the most convenient module when another capability owns the responsibility.
---
# Architecture Boundary Review

Every implementation shall preserve documented dependency rules.
Verify where relevant:

• Domain does not depend on Infrastructure
• Domain does not depend on Presentation
• Application does not depend on Presentation
• Presentation does not access concrete broker adapters directly
• provider models remain in Infrastructure
• persistence models remain in Infrastructure
• SQLAlchemy types do not leak into Domain
• PySide6 types do not leak into Domain
A technically working implementation may still be architecturally invalid.
---
# Architecture Decision Process

Before implementing an architectural change:
1. Verify whether an Architecture Decision Record already exists.
2. Review `Architecture.md` and affected capability documentation.
3. Identify the concrete product or technical requirement.
4. Evaluate existing architecture boundaries.
5. Evaluate real consumers.
6. Evaluate migration impact.
7. Define the smallest architecture change.
8. Update architecture documentation.
9. Create or update an ADR where the decision requires durable rationale.
10. Implement only after the architecture direction is clear.
Architectural changes shall remain consistent with the current documented architecture.
Do not introduce architecture solely for hypothetical future scalability.
---
# Implementation Rules

Every implementation shall:
• solve the defined requirement
• preserve architecture boundaries
• remain strongly typed
• make side effects explicit
• preserve deterministic state transitions
• remain independently testable
• use existing project abstractions where appropriate
• avoid unnecessary abstractions
Do not introduce a new service, port, repository or framework abstraction unless a concrete capability requires it.
---
# Code Quality

Mandatory:
• Python 3.13
• UTF-8
• `from __future__ import annotations`
• full type hints
• Ruff compliant
• explicit naming
• focused functions
• focused classes
Code shall follow `Coding_Standards.md`.
Code quality rules apply equally to:
• Domain
• Application
• Infrastructure

• Presentation
• scripts
• tests
---
# Business-Critical Changes

A change is business-critical when it may affect:
• trading decisions
• order creation
• order submission
• order cancellation
• execution processing
• position state
• reconciliation
• financial values
• PAPER or LIVE environment selection
Business-critical changes require explicit scenario analysis before implementation.
The analysis shall identify:
• previous behaviour
• intended behaviour
• invalid behaviour to prevent
• external side effects
• failure state
• recovery or reconciliation impact
---
# External Side Effects

External side effects require explicit ownership.
Examples:
• broker order submission
• broker order cancellation
• external API mutation
• persistent business state mutation
Before changing side-effect code verify:
1. Which Application workflow requests the side effect?
2. Which adapter executes the technical operation?
3. Which state exists before the operation?
4. Which state exists after local acceptance?
5. Which state requires external acknowledgement?
6. What happens on timeout?
7. What happens on disconnect?
8. What happens when external state is unknown?
Do not hide external side effects inside generic helper functions.
---
# Order Submission Changes

Order submission changes are trading-critical.
Before implementation verify the complete lifecycle:
```text
Submission Requested
↓
Validated
↓
Transmitted
↓
Acknowledged or Rejected
```

↓
Partially Filled or Filled
```

Do not collapse multiple states into one boolean or generic `submitted` state.
A successful adapter method call does not automatically mean broker acknowledgement.
Broker-derived acknowledgement state shall remain explicit.
---

# Duplicate Submission Risk

Every order submission change shall evaluate duplicate execution risk.
Review:
• stable order identity
• submission identity
• repeated callbacks
• repeated broker messages
• timeout behaviour
• disconnect behaviour
• reconnect behaviour
• application restart behaviour
• recovery behaviour
Do not blindly repeat order submission after unknown external state.
Duplicate-prevention rules belong to the Application workflow.
Infrastructure adapters shall not independently decide that resubmission is safe.
---

# Retry Review

Before adding retry behaviour evaluate:
• operation idempotency
• external side effects
• duplicate execution risk
• provider semantics
• workflow ownership
Automatic retry may be appropriate for selected read operations.
Automatic order submission retry is prohibited unless explicitly defined and approved by the owning order workflow.
Generic retry decorators shall not be added around trading-critical side-effect operations.
---

# PAPER and LIVE Impact

Every trading-related change shall evaluate PAPER and LIVE impact.
Verify:
• active execution environment remains explicit
• PAPER configuration remains PAPER
• LIVE requires explicit selection
• no fallback activates LIVE
• environment context remains visible
• tests do not require LIVE trading
A change that behaves differently in PAPER and LIVE shall document the difference explicitly.
---

# Broker Integration Changes

Before changing broker integration code review:
• provider communication
• connection lifecycle
• provider state translation
```

• provider identifier translation
• provider error translation
• acknowledgement semantics
• execution semantics
Broker adapters shall not contain:
• trading decisions
• candidate acceptance rules
• portfolio risk decisions
• duplicate submission policy
• position close decisions
Reconnection and order submission are separate workflows.
---
# Market Data Changes

Before changing market data code review:
• provider communication
• connection state
• source timestamps
• unavailable values
• stale data state
• subscription ownership
• subscription closure
Market data adapters shall not contain trading decision rules.
Cached data shall not silently appear as current provider data.
Missing financial data shall not silently become zero.
---
# Persistence Changes

Before changing persistence code review:
• Domain identity
• repository contract
• transaction ownership
• persistence model translation
• existing data compatibility
• reconciliation impact
Persistence models shall remain in Infrastructure.
Repository interfaces shall not expose SQLAlchemy models.
Do not silently rewrite business history during migration or recovery.
---
# Configuration Changes

Before changing configuration:
1. Identify the owning capability.
2. Identify the real consumer.
3. Define configuration source.
4. Define precedence.
5. Define type.
6. Define validation.
7. Evaluate PAPER impact.
8. Evaluate LIVE impact.
9. Evaluate secret handling.
10. Update tests.
11. Update documentation.
Configuration compatibility shall be evaluated based on real consumers.
Do not preserve obsolete internal configuration solely for hypothetical backward compatibility.
Unused configuration shall be removed.

---
# Runtime Changes

Before changing runtime behaviour review:
• startup order
• shutdown order
• runtime state
• task ownership
• cancellation
• timeout behaviour
• degraded state
• broker lifecycle
• market data lifecycle
• UI thread impact
• AsyncIO event loop impact
Long-running tasks require explicit lifecycle ownership.
Avoid unmanaged background tasks.
---
# Monitoring and Logging Changes

Monitoring and logging observe behaviour.
They shall not:
• submit orders
• cancel orders
• retry order submissions
• repair Domain state
• resolve reconciliation discrepancies
Monitoring shall not reconnect external systems independently.
Recovery actions require explicit runtime or application ownership.
Logging changes shall preserve canonical event naming and relevant business identity.
---
# Refactoring

Refactoring should:
• improve readability
• reduce complexity
• remove duplication
• clarify ownership
• preserve intended behaviour
Large refactorings should be split into small focused commits.
Do not combine behaviour changes and large structural refactoring without a concrete reason.
When behaviour must change during refactoring, document the intended behaviour change explicitly.
---
# Backward Compatibility

Backward compatibility shall be evaluated based on real consumers.
Preserve compatibility when:
• an independent consumer exists
• a public contract exists
• persisted state requires migration
• a documented integration depends on the contract
Do not preserve obsolete internal APIs or configuration solely for hypothetical future compatibility.
Breaking changes require:
• identified affected consumers

• migration plan where required
• automated test updates
• documentation updates
---
# Testing Strategy

Every code change requires appropriate testing.
Testing depth depends on risk.
Typical sequence:
1. Focused unit tests
2. Affected integration tests
3. Trading workflow regression tests where relevant
4. Runtime or system tests where relevant
5. Full required quality gates
Do not run only broad test suites when a focused failing scenario can first verify the change.
---
# Unit Tests

Use unit tests for:
• Domain rules
• state transitions
• Value Objects
• Application workflow decisions
• validation
• error mapping logic
Unit tests shall remain deterministic.
Use controlled inputs and explicit expected state.
---
# Integration Tests

Use integration tests for:
• repositories
• SQLite persistence
• configuration loading
• dependency registration
• broker adapter translation
• market data adapter translation
• runtime service coordination
Integration tests shall not require LIVE trading.
Use fake or controlled external boundaries where appropriate.
---
# Trading Workflow Regression Tests

Trading-critical changes require regression tests for the affected scenario.
Potential scenarios include:
• order validation
• duplicate submission prevention
• broker acknowledgement distinction
• broker rejection
• partial fill
• fill
• repeated broker messages
• timeout
• disconnect
• reconnect

• position lifecycle
• reconciliation discrepancy
A bug fix should reproduce the previous failure before verifying the correction whenever practical.
---
# Deterministic Tests

Tests shall avoid uncontrolled dependency on:
• wall-clock time
• network state
• LIVE broker services
• random data
• developer-local configuration
Use where required:
• controlled clocks
• fake adapters
• deterministic fixtures
• explicit test configuration
A test that depends on current market state is not a deterministic regression test.
---
# Test Failure Handling

A failing test shall be investigated.
Do not:
• delete the test solely to make CI pass
• weaken assertions without understanding the failure
• add arbitrary sleeps
• mark trading-critical tests skipped without explicit reason
Flaky tests shall be identified and stabilized.
Trading-critical flaky tests require priority investigation.
---
# Source Quality Verification

Required source quality checks:
```bash
ruff check .
ruff format --check .
```
CI verifies formatting but does not rewrite source files.
To apply formatting locally:
```bash
ruff format .
```
After formatting, rerun the affected tests.
---
# Documentation

Documentation is part of the implementation.
Update documentation whenever the change affects:
• architecture
• behaviour
• APIs
• configuration
• runtime lifecycle
• workflows
• operational state

• development process
Documentation updates occur before final verification, commit and push.
---
# Documentation Source of Truth

Markdown under:
```text
docs/
```
is the documentation source of truth.
Generated files under:
```text
docs/generated/docx/
docs/generated/pdf/
```
are derived artifacts.
Generated DOCX and PDF files shall not be edited manually.
---
# Documentation Generation

When Markdown documentation changes, run:
```bash
python scripts/generate_docs.py
```
The command shall complete successfully before the documentation change is considered verified.
Documentation validation operates on Markdown source.
Generated DOCX and PDF files are not re-read for content validation.
---
# Documentation Consistency

Before completing a change verify that affected documents remain mutually consistent.
Typical consistency relationships include:
```text
Architecture
↔ Infrastructure
↔ API Guidelines
Configuration
↔ Runtime
↔ CI/CD
Domain Model
↔ Coding Standards
↔ Testing Strategy
Logging
↔ Monitoring
↔ Runtime
```
Do not update one documented contract while leaving directly dependent documentation knowingly inconsistent.
---
# Git Workflow

Preferred development sequence:
```text
Analyze
↓
```

Implement
↓
Test
↓
Update Documentation
↓
Verify Quality Gates
↓
Review
↓
Commit
↓
Push
```

Git workflow details are defined in `Git_Workflow.md`.
---
# Commit Preparation

Before committing verify:
• requirement implemented
• focused tests passed
• required regression tests passed
• Ruff passed
• formatting check passed
• documentation updated
• documentation generation passed where required
• Project Analysis Agent checked where relevant
• no secrets added
• diff reviewed
Do not commit known incomplete business-critical behaviour as completed work.
---
# Commit Scope

Commits should be:
• small
• atomic
• descriptive
One commit should represent one coherent change.
Avoid combining:
• unrelated bug fixes
• unrelated refactoring
• documentation cleanup unrelated to the implementation
when separate commits would provide clearer history.
---
# Push

Push only after local verification is complete.
A successful push does not replace CI validation.
If CI fails:
1. identify the failed quality gate
2. reproduce locally where practical
3. correct the root cause
4. rerun relevant local verification
5. commit the correction
6. push again
Do not bypass required CI gates.
```

---

# Project Analysis Agent Quality Gate

The Project Analysis Agent provides local and CI-supported repository checks.
Local text report:
```bash
python tools/project_analysis_agent.py .
```
Local JSON report:
```bash
python tools/project_analysis_agent.py . --json
```
Local critical quality gate:
```bash
python tools/project_analysis_agent.py . --fail-on-critical
```
The critical quality gate fails on:
• missing important documentation
• empty Markdown files
• placeholder Markdown files
• architecture import violations
• Python parse errors
Trading safety hotspots are reported for review.
They are intentionally not CI-failing findings at this stage.
When CI fails because of the Project Analysis Agent:
1. read the critical finding
2. reproduce locally with `--fail-on-critical`
3. correct the root cause
4. rerun the agent
5. rerun affected tests or documentation generation
6. commit and push the correction
Do not silence the quality gate without understanding the finding.
---

# Definition of Review

Every implementation shall be reviewed for:
• requirement correctness
• architecture compliance
• capability ownership
• coding standards
• state transition correctness
• external side effects
• duplicate submission risk where relevant
• retry behaviour where relevant
• PAPER and LIVE impact where relevant
• persistence impact
• reconciliation impact
• async lifecycle
• error handling
• sensitive data exposure
• test coverage
• documentation consistency
• maintainability
Review the complete diff, not only the final edited function.
---

# Security Review

Before completion verify:
• no secrets committed
• no credentials exposed
• configuration remains secure
• logs do not expose sensitive information
• exceptions do not expose secrets
• generated documentation does not contain secrets
• CI does not require LIVE trading credentials
Sensitive provider payloads shall not be added to diagnostics without explicit review.
---
# Performance Review

Evaluate whether the implementation:
• blocks the UI thread
• blocks the AsyncIO event loop
• introduces unnecessary allocations
• increases startup time
• increases runtime complexity
• performs redundant operations
• creates excessive structured logging
• creates uncontrolled polling
Optimize only when measurable.
Performance optimization shall not weaken business correctness.
---
# Release Workflow

Before releasing:
1. Required CI quality gates pass.
2. Trading-critical regression tests pass.
3. PAPER and LIVE safety tests pass where applicable.
4. Documentation is updated.
5. Documentation generation succeeds.
6. Version is updated where applicable.
7. Changelog is updated.
8. Review is complete.
9. Changes are committed.
10. Changes are pushed.
Release workflow shall not require LIVE order execution.
---
# Pull Request Checklist

Before merging verify:
• requirement implemented
• tests passed
• trading regression tests passed where relevant
• Ruff passed
• formatting verification passed
• architecture boundaries respected
• capability ownership remains clear
• external side effects reviewed
• duplicate submission risk reviewed where relevant
• retry behaviour reviewed where relevant
• PAPER and LIVE impact reviewed where relevant

• documentation updated
• documentation generation passed where required
• no unnecessary dependencies added
• no duplicated logic introduced
• no secrets exposed
---

# Definition of Done

A task is complete only when:
• requirement is implemented
• implementation is reviewed
• focused tests pass
• required regression tests pass
• source quality checks pass
• documentation is synchronized
• documentation generation passes where required
• security impact is reviewed
• PAPER and LIVE impact is reviewed where relevant
• complete diff is reviewed
• changes are committed
• changes are pushed
A task with known unresolved trading-critical behaviour is not complete.
---

# Development Review Checklist

Before completing any implementation verify:
• requirement understood
• current implementation inspected
• related documentation reviewed
• owning capability identified
• architecture boundaries preserved
• smallest meaningful change implemented
• side effects explicit
• state transitions deterministic
• unavailable financial data remains explicit
• async task ownership defined
• cancellation evaluated
• timeout behaviour evaluated
• order lifecycle evaluated where relevant
• duplicate submission risk evaluated where relevant
• retry behaviour evaluated where relevant
• PAPER and LIVE impact evaluated where relevant
• persistence impact evaluated
• reconciliation impact evaluated
• tests added or updated
• focused tests passed
• required regression tests passed
• Ruff passed
• formatting check passed
• documentation synchronized
• documentation generation passed where required
• security reviewed
• complete diff reviewed
• committed
• pushed
---

# Related Documents

- Product_Vision.md
- Product_Roadmap.md
- Project_Overview.md
- Architecture.md
- Domain_Model.md
- Infrastructure.md
- Technical_Specifications.md
- API_Guidelines.md
- Coding_Standards.md
- Configuration.md
- Runtime.md
- Logging.md
- Monitoring.md
- CI_CD.md
- Git_Workflow.md
- Testing_Strategy.md
- AGENTS.md